

Enunciado Laboratório 03 de Compiladores.
Entrega 23/05/2026

Você deve:

- **Fazer o trabalho individualmente;**
- **Enviar um arquivo .zip contendo todos os arquivos necessários para rodar o analisador semântico em cima da fatia definida da sua gramática;**
- **Realizar os envios SOMENTE pelo Classroom, até 23/05/2026.**

Adapte o programa anexo para que o programa final faça a análise semântica de uma função na sua linguagem. Seu programa final deve receber um arquivo de programa escrito na sua linguagem sorteada, gerar um objeto em memória que descreve a função (tal como pode ser visto no exemplo em src/src-gram7/Funcao.hpp). Este objeto deverá ser tratado pela função calcula_retorno. Esta função deve simular uma execução da função e retornar o último valor atribuído no último comando.

Para simplificar esta fase, seu programa deve se restringir a um subconjunto da sua linguagem. Este subconjunto deve compreender as seguintes estruturas:

- Declaração de função, com parâmetros e corpo de função
- Declaração de variáveis, com o tipos restritos **a pelo menos:**
 - **um tipo inteiro**
 - **um tipo ponto flutuante**
 - **um tipo booleano ou seu substituto na linguagem.**
- Bloco de comandos, com possível declaração de novas variáveis, caso sua linguagem permita.
- Comando de atribuição
- Expressões com operadores aritméticos, incluindo + - * / % , variáveis, números inteiros, números ponto flutuante e booleanos.
- Expressões com operadores relacionais, incluindo "<", "<=", ">", ">=" e "=", "!=" , podendo incluir outros
- Expressões com operadores lógicos, incluindo &&, || e ! (e, ou e não).
- IF-THEN-ELSE
- WHILE, podendo incluir FOR, DO-WHILE e outros.

Seu analisador deve apresentar mensagem de erros se os tipos dos operandos de algum operador forem incompatíveis. Por exemplo, deve haver um erro se for feito "! ponto flutuante". Também deve haver erro para (booleano < inteiro) e para (inteiro || ponto flutuante), caso sua linguagem o faça. Aqui você deve gerar erro sempre que sua linguagem o fizer.

Seu analisador deve fazer conversões de tipos coerentemente com as regras da sua linguagem. Por exemplo, em C caso seja feito uma soma de inteiro e ponto flutuante, o resultado é um ponto flutuante.

No final, você deve testar o seu analisador passando exemplos de listas de parâmetros para sua função e imprimindo o último valor atribuído. Se o valor a ser impresso for booleano, seu programa deve imprimir "true" ou "false". Se o

valor for ponto flutuante, ele deve imprimir o número com 2 casas decimais. Se o número for inteiro, ele deve ser impresso da forma usual (decimal).

Note que foi feito um exemplo de conversão para objeto do tipo `Funcao` a partir de uma linguagem restrita a inteiros descrita no arquivo `gramatica.conf`. Você deve fazer a sua conversão a partir da declaração de uma função na sua linguagem, com as restrições acima.

NOTA: Consertei o parser de gramática. Agora somente deve ser usado o `gramatica9.site` e não mais o `gramatica.conf`.

Use `make run` para rodar.

Você deve criar um diretório `ins/` em `lab3-atual`. Você deve converter cada um dos exemplos de código C, nomeado `X.c` por exemplo, para sua linguagem, obtendo `X.ling`. Em seguida, a fase 1 léxica deve ser rodada sobre o código `X.ling` do exemplo e o arquivo de tokens `X.tokens` criado. Em seguida, exemplos de parâmetros devem ser passados e descritos em um arquivo de nome `X.params`. O resultado da execução do `make run` sobre `X.tokens` com os parâmetros de `X.params` deve ser colocado na saída `X.saida`. Todos estes arquivos devem ser colocados no diretório `ins/`.

Para a submissão você deve remover os arquivos compilados (`.o` e executáveis) e comprimir todo o diretório com sua solução e com o diretório `ins/` e submeter somente este `.zip`.